Youdahe Asfaw

| youdaheasfaw@gmail.com | linkedin.com/in/youdahe | github.com/youdahe |

Education

Minnesota State University Mankato June 2028
Bachelor of Science in Computer Science & Statistics ***3.5/4.0***

- **Relevant Coursework:** Introduction to Data, Object-Oriented Programming, Data Structures and Algorithms

Technical Skills

Languages: Python, TypeScript, Java, JavaScript, HTML/CSS, Go, Rust
Frameworks: FastAPI, LangChain, Tesseract.js, React.js, Express, GraphQL, Spring, Three.js
Developer Tools: Git, Selenium, Postman, Node.js, Jenkins, MongoDB, Docker, AWS, SQL (MySQL), Kubernetes, Cassandra, Ansible

Experience

Member of Technical Staff Intern Aug 2026 – Nov 2026
NMDP *Minneapolis, MN*

- Incoming @ CIBMTR working on Stem Cell Data Infrastructure + Agentic Research Systems

Software Engineer Intern May 2026 – Aug 2026
Liberty Mutual Insurance *Portsmouth, NH*

- Architecting a self-service internal tooling platform enabling one-click retirement of legacy AIX-hosted IBM MQ brokers, automating decommissioning workflows across distributed messaging infrastructure and **saving hundreds of thousands annually** in AIX licensing and infrastructure overhead.
- Driving migration of on-prem MQ workloads to Kubernetes-native deployments by designing **StatefulSets, PVC-backed durable message storage, and CNI-configured pod networking** with namespace-isolated broker topologies and strict east-west traffic policies.

Software Engineer Intern Jan 2026 – May 2026
Medica *Minnetonka, MN*

- Engineered an internal **RAG pipeline** from the ground up — designing document ingestion and chunking strategy, embedding generation via transformer-based encoders, and vector store indexing with metadata filtering for retrieval precision.
- Built a context assembly layer with **hybrid sparse-dense retrieval, reranking, and prompt construction pipelines** feeding a stateful AI agent grounded against live internal knowledge, reducing employee onboarding time **from one week to under two days**.

Machine Learning Research Assistant Feb 2025 – Jun 2025
Gustavus Adolphus College *Saint Peter, MN*

- Built an ML-powered, agentic voice-guided form-completion system using **Python, Azure Form Recognizer, and LayoutLMv3**, reducing completion time from 60 to 15 minutes.
- Improved form accessibility for visually impaired users, enabling faster and more accurate submissions via automated email or print outputs.

Software Engineer Intern Aug 2023 – Aug 2024
Medtronic *Mounds View, MN*

- Built and maintained **distributed data pipelines** across the Touch Surgery ecosystem — ingesting high-volume intraoperative surgical video feeds, instrument telemetry, and case metadata into scalable backend infrastructure, with ETL pipelines routing data into AI-driven analytics for surgical workflow and instrument detection.
- Worked across **PostgreSQL and .NET-based services** to ensure data reliability and consistency across case management, video storage, and benchmarking layers, coordinating 12+ user acceptance tests and increasing processing efficiency by **30%** for teams tracking **over $2.5M** in R&D projects.

Projects

Memory-Augmented AI Tutoring Platform | *TypeScript, SQL,Redis, Docker, Kubernetes*

- Made Docere a **memory-augmented AI tutoring platform** serving **100+ students across 2 schools**, featuring a three-source RAG layer (Qdrant vector search, PostgreSQL student profiles, interaction history) with a UCB1 multi-armed bandit for teaching strategy selection and Bayesian Knowledge Tracing for per-concept mastery — improving AI response relevance by **60%** and task completion by **40%**.
- Built a full-stack async pipeline (FastAPI, React 19, Redis/ARQ, LangGraph) with dual LMS adapters (Canvas + Moodle via LTI 1.3) and GitHub Actions CI/CD, **securing $10,000 in funding** and launching pilots ingesting real gradebooks, course materials, and calendar scheduling.

Embedded LSM Storage Engine | *Rust, Cargo*

- Built youdaheDB a distributed key-value database engine in Rust implementing a **Write-Ahead Log** for crash recovery, an in-memory MemTable backed by a BTreeMap for sorted key lookups, and SSTable-based persistent storage with custom binary serialization using length-prefixed encoding.
- Designed an **LSM-tree storage architecture** with durable write path (WAL → MemTable → SSTable flush), tombstone-based deletes, and WAL replay for fault-tolerant recovery — leveraging Rust's ownership model for compile-time memory safety with zero-copy reads and buffered I/O.